Argentor

Framework Seguro de Orquestación Multi-Agente con IA en Rust

Reporte Técnico — Versión 0.1.0 | Abril 2026

3.400	~130K	15	14	40+
Tests	Líneas de código	Crates	Proveedores LLM	Skills integradas

0

Advertencias clippy

Tabla de Contenidos

1. Resumen Ejecutivo
2. Arquitectura General
3. Mapa de Crates
4. El Bucle Agéntico
5. Orquestación Multi-Agente
6. Arquitectura de Seguridad

7. Sistema de Plugins WASM
8. Soporte de Protocolos (MCP + A2A)
9. Skills Integradas (40+)
10. Gateway y API REST
11. Stack de Observabilidad
12. Módulos de Cumplimiento Normativo
13. Capa de Datos y Memoria
14. SDKs y Bridges de Lenguajes
15. Despliegue
16. Métricas del Proyecto

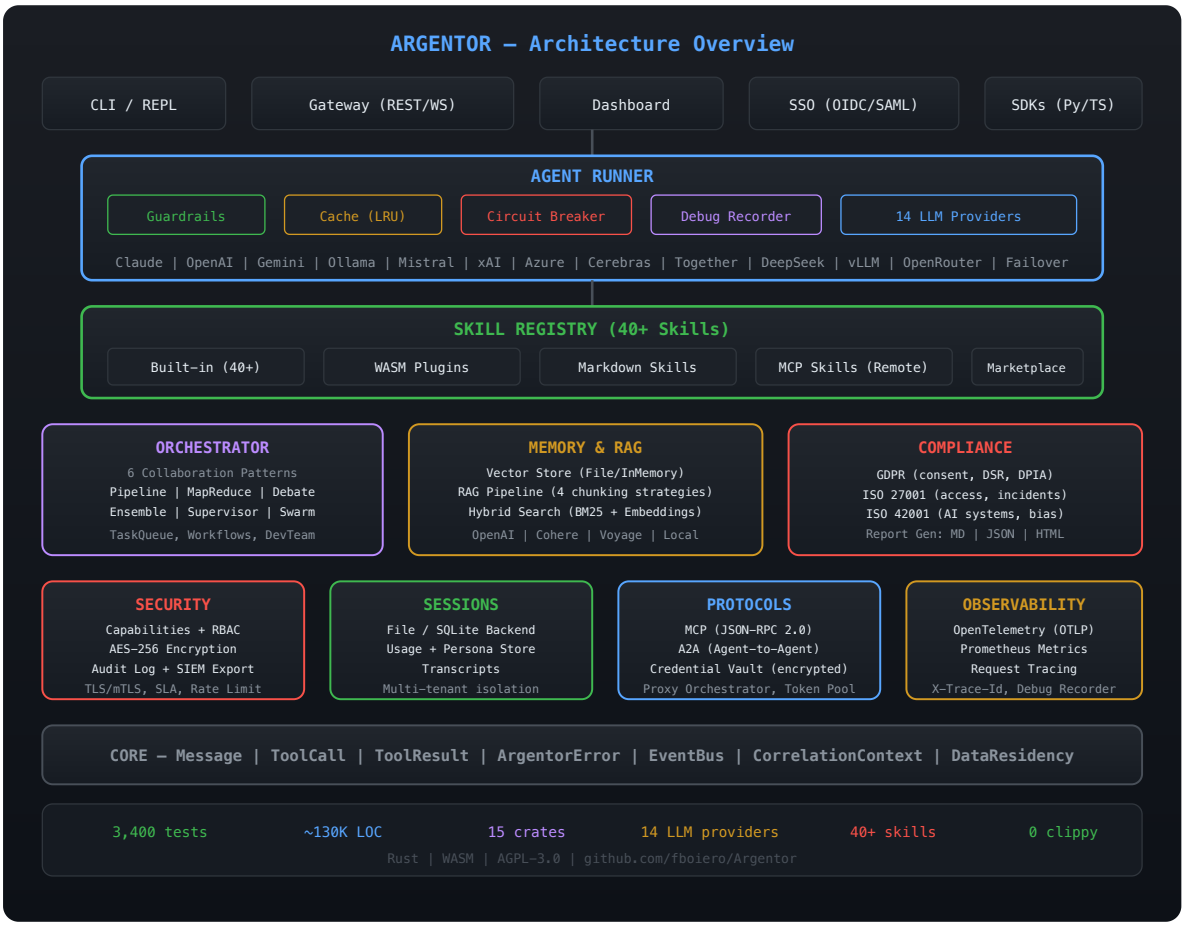
1. Resumen Ejecutivo

Argentor es un framework de orquestación multi-agente de IA de grado productivo, desarrollado en Rust. Proporciona:

- **15 crates** con capas de dependencia limpias (cero dependencias circulares)
- **14 proveedores de LLM** (Claude, OpenAI, Gemini, Ollama, Mistral, xAI, Azure, Cerebras, Together, DeepSeek, vLLM, OpenRouter, Claude Code, Failover)
- **40+ skills integradas** que cubren procesamiento de datos, búsqueda web, criptografía, escaneo de seguridad, operaciones de archivos y análisis de código
- **Plugins WASM aislados** con firma Ed25519 y permisos basados en capacidades
- **Soporte de protocolos MCP + A2A** para interoperabilidad entre agentes
- **Pipeline de guardrails** integrado en el bucle agéntico (PII, inyección, toxicidad)
- **Módulos de cumplimiento empresarial** (GDPR, ISO 27001, ISO 42001, DPGA)
- **3.400 tests**, cero fallos, cero advertencias de clippy

2. Arquitectura General

El diagrama siguiente muestra la arquitectura completa de Argentor, desde las interfaces de usuario (CLI, Gateway, Dashboard) hasta las capas fundacionales (Core, Security, Sessions). Cada componente se implementa como un crate independiente con responsabilidades bien definidas.



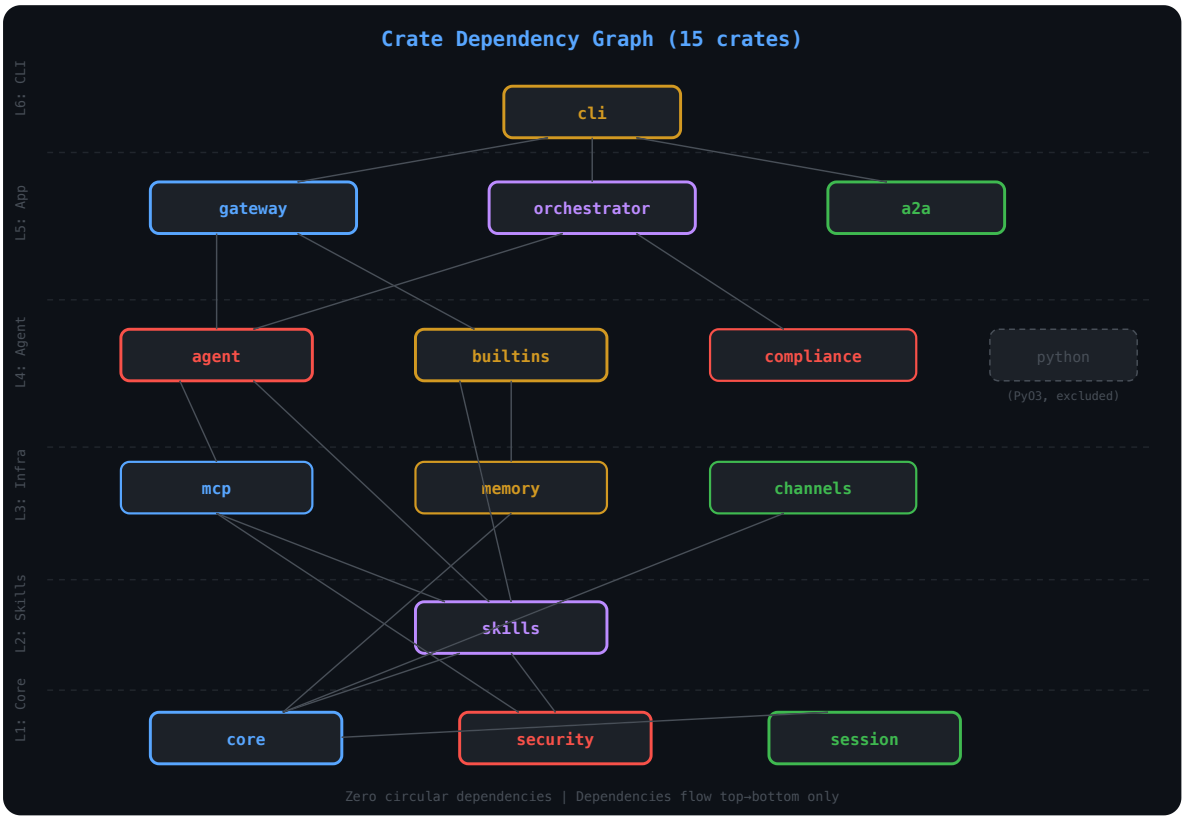
El framework se estructura en capas bien delimitadas:

- **Capa de presentación:** CLI/REPL, Gateway HTTP/WebSocket, Dashboard HTML y autenticación SSO (OIDC/SAML)
- **Capa de agente:** AgentRunner con guardrails, caché LRU, circuit breaker, grabador de depuración y 14 backends LLM
- **Capa de habilidades:** Registro de skills con 40+ integradas, plugins WASM, skills Markdown y skills MCP remotas
- **Capa de orquestación:** Motor multi-agente con 6 patrones de colaboración, memoria RAG con búsqueda híbrida y módulos de cumplimiento normativo

- **Capa de infraestructura:** Seguridad (RBAC, cifrado AES-256, auditoría), sesiones (SQLite/archivo), protocolos (MCP, A2A) y observabilidad (OpenTelemetry, Prometheus)
- **Capa fundacional:** Tipos core (Message, ToolCall, ToolResult), bus de eventos, contexto de correlación y residencia de datos

3. Mapa de Crates

Argentor se compone de 15 crates organizados en 6 capas de dependencia. Las dependencias fluyen exclusivamente de arriba hacia abajo, sin ciclos circulares.

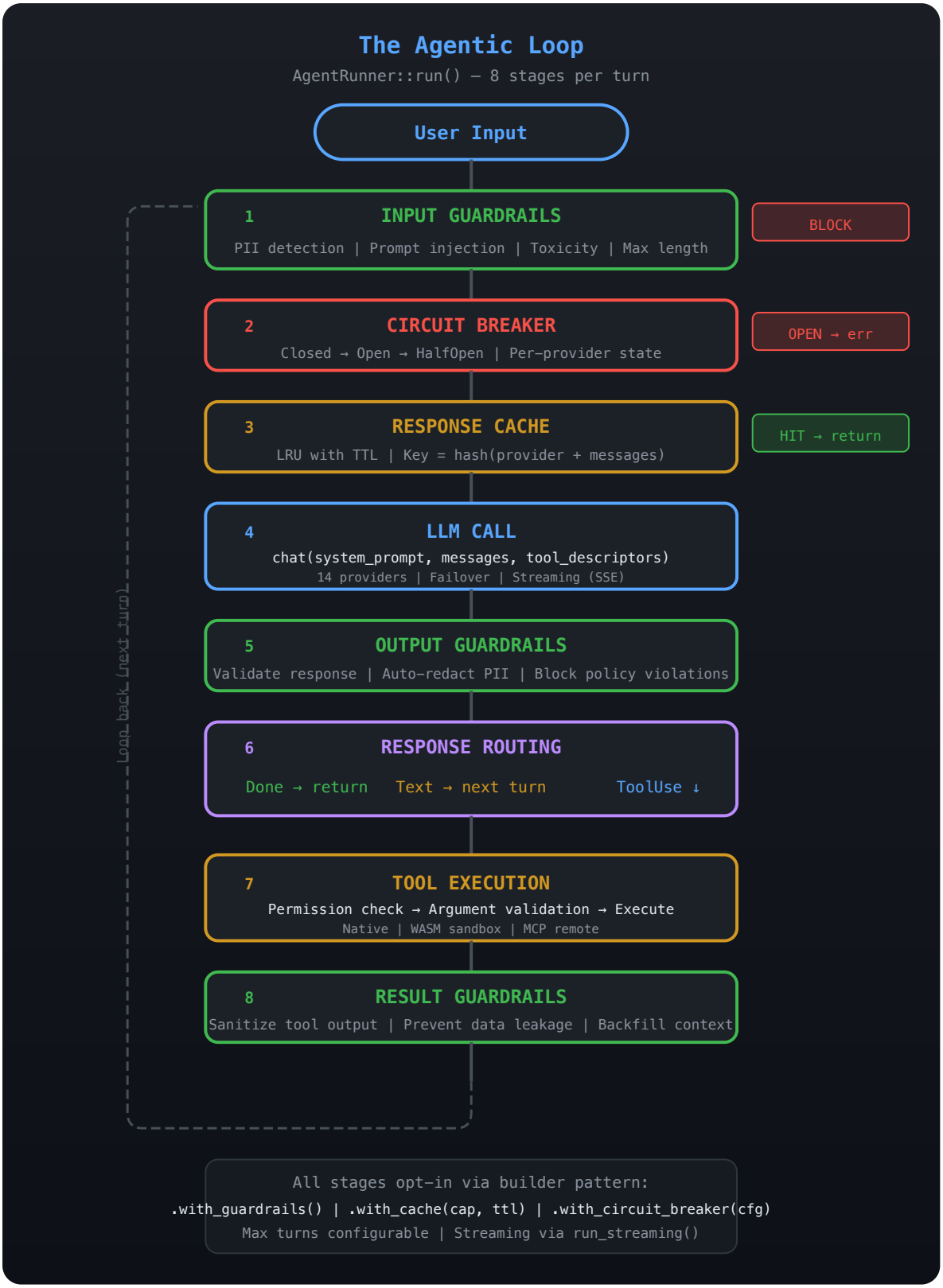


Descripción de Crates

#	Crate	LOC	Tests	Propósito
1	`argentor-core`	~8K	63	Tipos (Message, ToolCall, ToolResult), bus de eventos, correlación, telemetría
2	`argentor-security`	~12K	526	Capacidades, RBAC, cifrado AES-256, registro de auditoría, SLA, limitación de tasa
3	`argentor-session`	~5K	84	Ciclo de vida de sesión, backend SQLite, transcripciones, almacenes de uso/persona
4	`argentor-skills`	~8K	170	Trait Skill, runtime WASM (wasmtime), registro, marketplace, vetting
5	`argentor-agent`	~18K	900	AgentRunner, 14 backends LLM, guardrails, ReAct, evaluador, caché
6	`argentor-channels`	~3K	18	Trait Channel, adaptador Slack, WebChat, gestor
7	`argentor-gateway`	~20K	550	Axum HTTP/WS, API REST, SSO, observabilidad, API de marketplace
8	`argentor-builtins`	~25K	900	40+ skills integradas (todas las categorías)
9	`argentor-memory`	~8K	140	Almacén vectorial, pipeline RAG, BM25, embeddings (3 proveedores)
10	`argentor-mcp`	~6K	126	Cliente/servidor MCP, bóveda de credenciales, pool de tokens, orquestador proxy
11	`argentor-orchestrator`	~15K	356	Motor multi-agente, 6 patrones, workflows, despliegue, salud
12	`argentor-compliance`	~7K	280	GDPR, ISO 27001, ISO 42001, DPGA, generador de reportes
13	`argentor-a2a`	~4K	125	Protocolo Google A2A, tarjetas de agente, streaming SSE
14	`argentor-cli`	~5K	—	Binario, REPL, 8 demos, configuración
15	`argentor-python`	~1K	—	Bridge PyO3 (9 clases, compilación con maturin)
	Total	**~130K**	**3.400**	

4. El Bucle Agéntico

El motor de ejecución principal (`AgentRunner::run()`) implementa un bucle agéntico multi-turno con 8 etapas. Cada turno de conversación recorre estas etapas secuencialmente, con la posibilidad de iterar cuando el LLM solicita la ejecución de herramientas.



Decisiones de diseño clave

- Los guardrails se ejecutan en 3 puntos: entrada pre-LLM, salida post-LLM y resultado post-herramienta

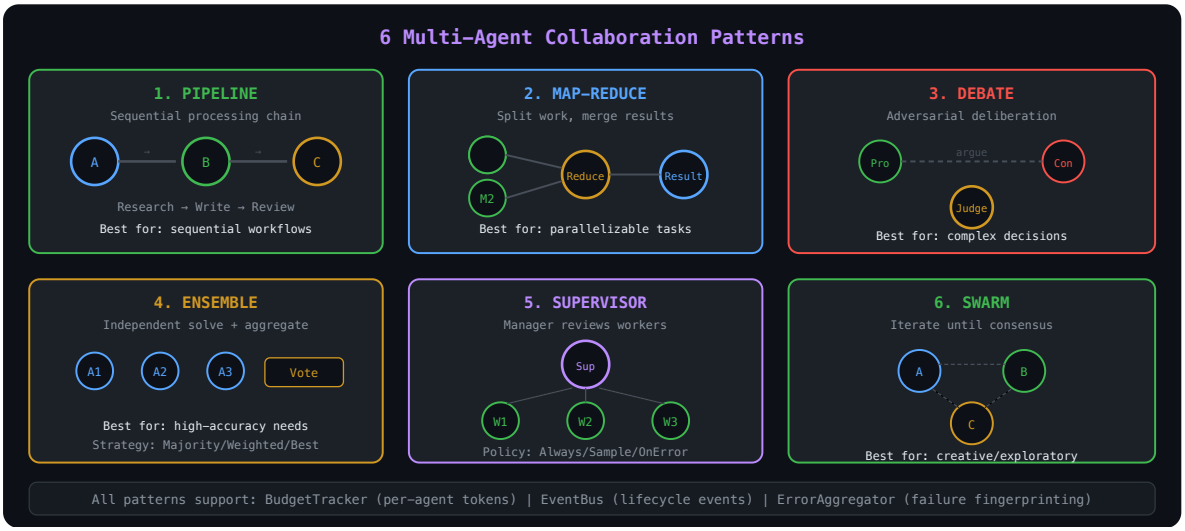
- La caché opera ANTES de los guardrails de salida (las respuestas en caché ya fueron validadas)
- El circuit breaker previene el desperdicio de tokens en proveedores con fallos
- Todas las etapas son opcionales y se activan mediante el patrón builder (`with_guardrails()`, `with_cache()`, etc.)

5. Orquestación Multi-Agente

El `Orchestrator` descompone tareas complejas en sub-tareas y las distribuye entre agentes trabajadores especializados. Cada agente posee su propio conjunto de skills y puede operar de forma independiente bajo la coordinación del orquestador.

El sistema emite eventos de ciclo de vida (`task.started` , `task.completed` , `task.failed`) a través del EventBus, utiliza un ErrorAggregator para el fingerprinting de fallos y un BudgetTracker para los límites de tokens por agente.

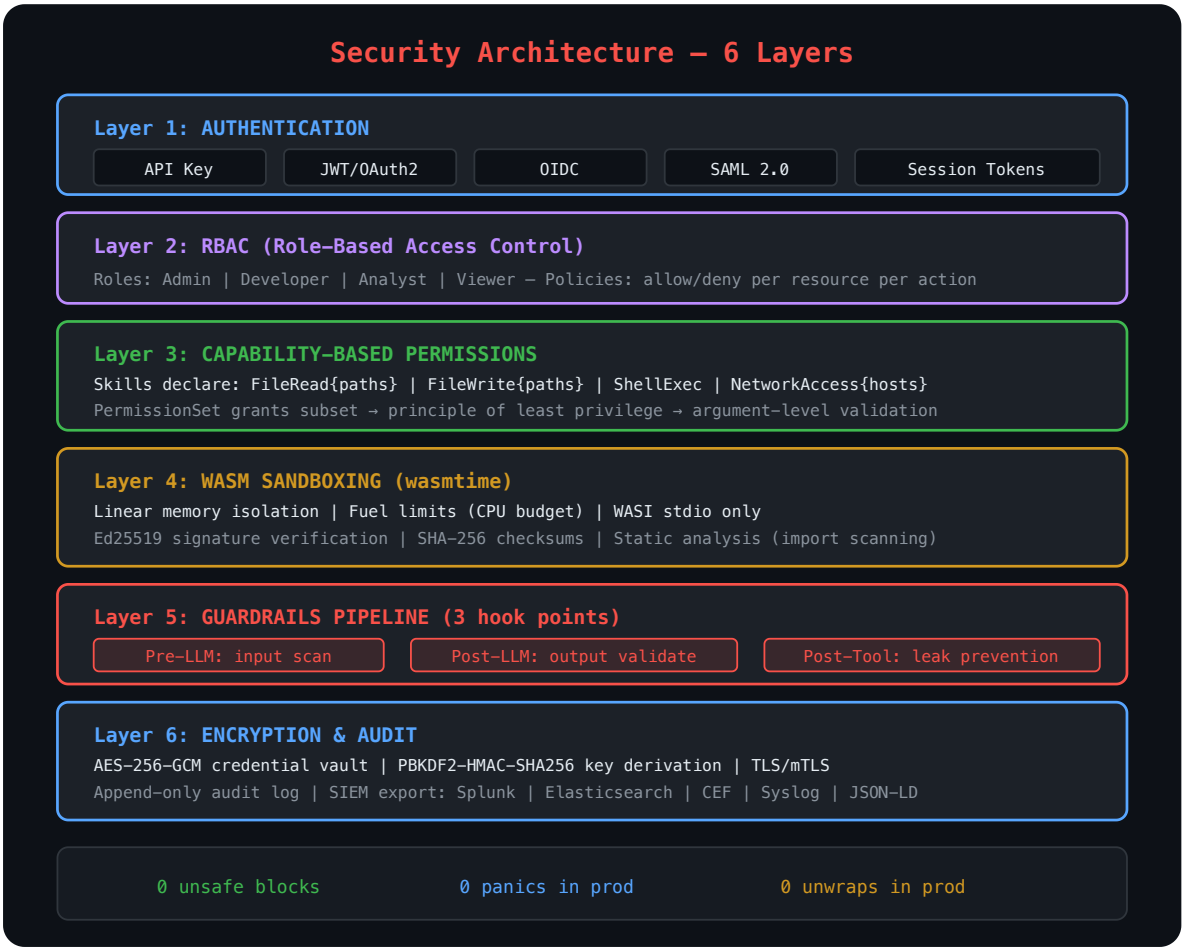
6 Patrones de Colaboración



Patrón	Ideal para	Ejemplo
Pipeline	Flujos de trabajo secuenciales	Investigar → Redactar → Revisar
MapReduce	Tareas paralelizables	Analizar 100 archivos, fusionar resultados
Debate	Decisiones complejas	Evaluación de seguridad con revisión adversarial
Ensemble	Necesidades de alta precisión	Múltiples agentes votan sobre clasificación
Supervisor	Aseguramiento de calidad	Un manager revisa todas las salidas de los workers
Swarm	Creatividad y exploración	Lluvia de ideas con convergencia

6. Arquitectura de Seguridad

Argentor implementa un modelo de seguridad en profundidad con 6 capas, desde la autenticación en el perímetro hasta el cifrado y auditoría de todas las operaciones. El diseño sigue el principio de mínimo privilegio en cada capa.



Resumen de las 6 capas

1. **Autenticación:** API Key, JWT/OAuth2, OIDC, SAML 2.0, tokens de sesión
2. **Autorización RBAC:** Roles (Admin, Developer, Analyst, Viewer) con políticas de permitir/denegar por recurso y acción
3. **Permisos basados en capacidades:** Cada skill declara las capacidades que requiere (FileRead, FileWrite, ShellExec, NetworkAccess, etc.). PermissionSet otorga un subconjunto, aplicando el principio de mínimo privilegio con validación a nivel de argumento
4. **Sandboxing WASM:** Los plugins se ejecutan en wasmtime con aislamiento de memoria lineal, límites de fuel (prevención de bucles infinitos), verificación de firma

Ed25519, validación de checksums SHA-256 y análisis estático de imports

5. **Pipeline de guardrails (3 puntos de intercepción):** Pre-LLM (detección de PII, inyección de prompts, toxicidad), Post-LLM (validación de salida, redacción automática de PII), Post-herramienta (prevención de fuga de datos)
6. **Cifrado y auditoría:** Bóveda de credenciales cifrada con AES-256-GCM, derivación de claves PBKDF2-HMAC-SHA256, registro de auditoría append-only, exportación SIEM (Splunk, Elasticsearch, CEF, Syslog, JSON-LD), TLS/mTLS

7. Sistema de Plugins WASM

El sistema de plugins de Argentor utiliza WebAssembly (WASM) para proporcionar extensibilidad con aislamiento total. Los desarrolladores compilan sus skills a `.wasm`, las firman con Ed25519, y el runtime verifica la integridad antes de ejecutarlas en un entorno restringido.

Flujo de trabajo del plugin

1. **Desarrollo:** El desarrollador escribe la skill en Rust y la compila a WASM
2. **Firma:** Se genera un manifiesto (`SkillManifest`) con nombre, versión, checksum SHA-256, firma Ed25519, capacidades requeridas, etiquetas y licencia
3. **Verificación (5 comprobaciones del Skill Vetter):**
   - Validación de checksum SHA-256
   - Límite de tamaño de binario (máximo 10 MB)
   - Verificación de firma Ed25519
   - Análisis de capacidades (lista de bloqueo)
   - Análisis estático WASM (número mágico + escaneo de imports)
4. **Ejecución en wasmtime:**
   - Aislamiento de memoria lineal (sin acceso al host)
   - Límites de fuel (presupuesto de CPU)
   - Solo WASI stdio
   - Sin acceso a sistema de archivos ni red
   - `spawn_blocking()` para seguridad asíncrona

8. Soporte de Protocolos (MCP + A2A)

Model Context Protocol (MCP)

Argentor implementa MCP tanto como cliente como servidor, utilizando JSON-RPC 2.0 como protocolo de transporte.

Como cliente MCP: Argentor se conecta a servidores MCP externos vía stdio, invocando métodos `tools/list` y `tools/call` para descubrir y ejecutar herramientas remotas.

Como servidor MCP: Argentor expone todas las skills registradas como herramientas MCP accesibles vía HTTP, permitiendo que clientes externos como Claude o Cursor las utilicen.

Infraestructura MCP:

- `McpProxy` : Multiplexa llamadas a múltiples servidores MCP
- `CredentialVault` : Almacenamiento cifrado de API keys con AES-256 y rotación
- `TokenPool` : Limitación de tasa por proveedor con prioridad por nivel
- `ProxyOrchestrator` : Enrutamiento (Fixed/RoundRobin/LeastLoaded/PatternBased), circuit breaker y failover

Protocolo Agent-to-Agent (A2A)

Argentor implementa el protocolo Google A2A para comunicación directa entre agentes, permitiendo la interoperabilidad con cualquier agente compatible con A2A.

Características de A2A:

- Descubrimiento de agentes vía `/.well-known/agent.json`
- Ciclo de vida de tareas: submitted → working → completed/failed/canceled
- Streaming SSE para actualizaciones de tareas en tiempo real
- Autenticación: Bearer, API Key, OAuth2

9. Skills Integradas (40+)

Argentor incluye más de 40 skills integradas organizadas en categorías funcionales. Estas skills cubren desde operaciones básicas de datos y texto hasta análisis de código, seguridad y orquestación multi-agente.

Categorías de Skills

Categoría	Skills
Datos y Texto	`calculator`, `text_transform`, `json_query`, `regex`, `data_validator`, `datetime`
Criptografía y Codificación	`hash` (SHA/HMAC), `encode_decode`, `uuid_generator`
Web y Búsqueda	`web_search` (DDG/Tavily/Brave/SearXNG), `web_scraper`, `rss_reader`, `dns_lookup`
Archivos y Sistema	`file_read`, `file_write`, `shell`, `http_fetch`
Código y Desarrollo	`git`, `code_analysis`, `test_runner`, `sdk_generator`, `api_scaffold`, `iac_generator`
Seguridad e IA	`prompt_guard`, `secret_scanner`, `diff`, `summarizer`
Memoria	`memory_store`, `memory_search`
Orquestación	`agent_delegate`, `task_status`, `human_approval`, `artifact_store`
Navegador y Docker (opcional)	`browser_auto`, `docker_sandbox`

Grupos de Herramientas (Divulgación Progresiva)

Grupo	Skills	Caso de uso
`minimal`	echo, time, help, calculator, datetime	Seguro para cualquier contexto
`coding`	file_read/write, shell, memory, regex, json_query, diff	Tareas de desarrollo
`web`	http_fetch, browser, web_search, web_scraper, rss_reader, dns_lookup	Tareas web
`data`	calculator, text_transform, json_query, regex, data_validator, encode_decode, hash, uuid, diff, summarizer	Procesamiento de datos
`security`	prompt_guard, secret_scanner, hash, data_validator, encode_decode	Escaneo de seguridad
`development`	git, code_analysis, test_runner + herramientas de coding	Flujo de desarrollo completo
`orchestration`	agent_delegate, task_status, human_approval, artifact_store	Multi-agente
`full`	Todas las skills registradas	Sin restricciones

10. Gateway y API REST

El servidor Gateway está construido sobre Axum e implementa una pila de middleware completa con trazabilidad de solicitudes, autenticación SSO, limitación de tasa por clave y validación de API key/JWT.

Pila de Middleware (orden de aplicación)

1. Trazabilidad de solicitudes (X-Trace-Id, spans)
2. Autenticación SSO (validación de token Bearer / cookie)
3. Limitación de tasa por clave (ventana deslizante, 429 + cabeceras)
4. Autenticación (validación de API key / JWT)

Endpoints

Grupo	Endpoint	Descripción
Salud	`GET /health`	Comprobación básica
	`GET /health/live`	Sonda de liveness
	`GET /health/ready`	Sonda de readiness
Auth	`GET /auth/login`	Redirección SSO
	`GET /auth/callback`	Callback SSO
	`GET /auth/logout`	Revocar sesión
	`GET /auth/me`	Usuario actual
	`POST /auth/api-key`	Clave → sesión
API de Agente `/api/v1/`	`POST /agent/chat`	Chat con agente
	`POST /agent/run-task`	Ejecutar tarea
	`POST /agent/run-task-stream`	Tarea con streaming (SSE)
	`POST /agent/batch`	Ejecución por lotes
	`POST /agent/evaluate`	Evaluar agente
	`GET /agent/status`	Estado del agente
Sesiones	`GET /sessions`	Listar sesiones
	`GET /sessions/:id`	Obtener sesión
	`DELETE /sessions/:id`	Eliminar sesión
Skills	`GET /skills`	Listar skills
	`GET /skills/:name`	Detalle de skill
Marketplace `/api/v1/marketplace/`	`GET /search`	Buscar skills
	`GET /featured`	Skills destacadas
	`GET /categories`	Categorías
	`GET /popular`	Skills populares
	`GET /recent`	Skills recientes
	`POST /install/:name`	Instalar skill
	`DELETE /install/:name`	Desinstalar skill

Grupo	Endpoint	Descripción
	`GET /stats`	Estadísticas
Observabilidad	`GET /metrics`	Métricas Prometheus
	`GET /openapi.json`	Especificación OpenAPI 3.0
	`GET /dashboard`	Dashboard HTML SPA
WebSocket	`WS /ws`	Comunicación bidireccional
Protocolo A2A	`GET /.well-known/agent.json`	Tarjeta del agente
	`POST /a2a`	JSON-RPC
	`POST /a2a/stream`	Streaming SSE

11. Stack de Observabilidad

El stack de observabilidad de Argentor proporciona visibilidad completa del comportamiento del sistema en producción, con cuatro componentes principales: trazabilidad distribuida, métricas, registro de auditoría y grabador de depuración.

Trazabilidad (OpenTelemetry)

Instrumentación con `#[tracing::instrument]` en los puntos clave del sistema:

- `AgentRunner::run()` (session_id, max_turns)
- `AgentRunner::execute_tool()` (tool_name, id)
- `AgentRunner::run_streaming()` (streaming=true)
- Llamadas LLM (provider, turn, session_id)

El middleware de solicitudes agrega cabeceras `X-Trace-Id` y `X-Span-Id` a las respuestas, junto con spans de método/ruta/estado/duración. La exportación se realiza vía OTLP gRPC (compatible con Jaeger, Tempo, Datadog).

Métricas (Prometheus)

`GET /metrics` expone las siguientes métricas:

Métrica	Etiquetas	Tipo
`http_requests_total`	method, path, status	Counter
`http_request_duration_seconds`	method, path	Histogram
`active_connections`	—	Gauge
`llm_calls_total`	provider	Counter
`tool_executions_total`	skill	Counter
`tokens_used_total`	provider, direction	Counter

Registro de Auditoría

Cada acción queda registrada con: session_id, acción, skill, resultado (Success/Denied/Error) y detalles en JSON. Formatos de exportación: Splunk, Elasticsearch, CEF, JSON-LD, Syslog.

Grabador de Depuración

Traza paso a paso del procesamiento: Input → LlmCall → ToolCall → ToolResult → LlmResponse → Output, con tiempos, conteos de tokens y metadatos asociados.

12. Módulos de Cumplimiento Normativo

Argentor integra cuatro marcos normativos de cumplimiento que pueden operar de forma independiente o combinada, con un generador de reportes unificado.

GDPR (Reglamento General de Protección de Datos)

- **ConsentStore:** Seguimiento de consentimiento con verificación de expiración
- **Procesamiento de solicitudes DSR (Data Subject Requests):**
  - Acceso a datos personales
  - Eliminación (derecho al olvido)
  - Portabilidad de datos
  - SLA de 30 días para todas las solicitudes

ISO 27001 (Seguridad de la Información)

- **Control de acceso:** Registro de eventos de inicio de sesión y tasa de denegaciones
- **Gestión de incidentes:** Severidad, resolución y ciclo de vida completo

ISO 42001 (Sistemas de IA)

- **Inventario de IA:** Nivel de riesgo e información del modelo
- **Verificaciones de sesgo:** Resultados de aprobado/advertencia/fallo
- **Transparencia:** Registro de decisiones del sistema de IA

DPGA (Alianza de Bienes Públicos Digitales)

- 9 indicadores de conformidad
- Verificación de código abierto
- Evaluación de privacidad de datos

Generador de Reportes

El generador de reportes integra datos de los cuatro marcos y produce salidas en tres formatos:

- **Markdown:** Para documentación y revisión

- **JSON:** Para integración con APIs
- **HTML:** Para dashboards y presentaciones

Cada reporte incluye: resumen ejecutivo con estado general (Conforme/Parcial/No conforme), puntuación por marco (0-100%), lista de hallazgos críticos, recomendaciones y fecha de próxima revisión.

13. Capa de Datos y Memoria

La capa de datos y memoria de Argentor proporciona almacenamiento persistente, búsqueda semántica y contexto conversacional entre sesiones.

Pipeline RAG

El pipeline completo de Retrieval-Augmented Generation sigue el flujo:

Ingestar → Fragmentar → Embeber → Almacenar → Consultar → Formatear

- **Estrategias de fragmentación:** Tamaño fijo, por párrafo, por oración, semántica
- **Proveedores de embedding:** Local (TF-IDF), OpenAI, Cohere, Voyage
- **Búsqueda:** Híbrida (similitud vectorial + BM25)
- **Expansión de consultas:** Sinónimos y términos relacionados basados en reglas

Almacenes Vectoriales

Dos implementaciones del trait `VectorStore` (extensible mediante traits):

- **InMemoryVectorStore:** Para desarrollo y testing
- **FileVectorStore:** Almacenamiento persistente en disco

Almacenamiento de Sesiones

- **FileSessionStore:** Almacenamiento basado en archivos
- **SQLite Backend:** Con modo WAL para concurrencia
- **JSON Index Store:** Índice para búsqueda rápida

Memoria Conversacional

Contexto por cliente entre sesiones:

- **CustomerProfile:** Temas de interés, sentimiento, historial
- **ConversationSummarizer:** Resúmenes con presupuesto de tokens
- **Inyección en system prompt:** Contexto personalizado automático

14. SDKs y Bridges de Lenguajes

Argentor ofrece tres vías de integración para desarrolladores: un SDK de Python, un SDK de TypeScript y un bridge nativo PyO3 para ejecución directa sin overhead HTTP.

SDK de Python (`pip install argentor-sdk`)

```
from argentor import ArgentorClient

client = ArgentorClient(base_url="http://localhost:8080", api_key="key")

# Ejecutar tarea del agente
result = client.run_task(role="researcher", context="Analyze market trends")

# Respuestas en streaming
for event in client.run_task_stream(role="writer", context="Write a report"):
    print(event.text, end="")

# Ejecutar skills directamente
result = client.execute_skill("calculator", {"operation": "evaluate", "expressi

# Verificación de salud
health = client.health_ready()
```

SDK de TypeScript (`npm install @argentor/sdk`)

```
import { ArgentorClient } from '@argentor/sdk';

const client = new ArgentorClient({ baseUrl: 'http://localhost:8080', apiKey: '

const result = await client.runTask({ role: 'analyst', context: 'Summarize Q4 re

for await (const event of client.runTaskStream({ role: 'writer', context: 'Draft
  process.stdout.write(event.text);
}
```

Bridge PyO3 (`maturin develop`)

```
import argentor

# Ejecución directa en Rust desde Python (sin overhead HTTP)
registry = argentor.SkillRegistry()
result = registry.execute("calculator", '{"operation": "evaluate", "expression"
print(result.content)  # "12"

guard = argentor.GuardrailEngine()
check = guard.check_input("My SSN is 123-45-6789")
print(check.passed)          # False
print(check.sanitized_text)  # "My SSN is [SSN]"
```

15. Despliegue

Docker para Producción

```yaml
# docker-compose.production.yml
services:
  argentor:
    build: .
    ports: ["8080:8080"]
    deploy:
      resources:
        limits: { memory: 512M, cpus: '2' }
    read_only: true
    cap_drop: [ALL]
    security_opt: [no-new-privileges:true]
    healthcheck:
      test: ["CMD", "wget", "--spider", "http://localhost:8080/health/live"]

  prometheus:   # opcional (--profile monitoring)
    image: prom/prometheus:latest
    ports: ["9090:9090"]

  grafana:      # opcional (--profile monitoring)
    image: grafana/grafana:latest
    ports: ["3000:3000"]
```

La imagen Docker de producción se ejecuta con las siguientes restricciones de seguridad:

- **read_only:** Sistema de archivos raíz en modo solo lectura
- **cap_drop: [ALL]:** Se eliminan todas las capabilities de Linux
- **no-new-privileges:** Previene la escalada de privilegios
- **Límites de recursos:** 512 MB de memoria, 2 CPUs
- **Health check:** Sonda de liveness integrada

Helm Chart (Kubernetes)

Se incluye un chart Helm completo para despliegue en Kubernetes:

```
deploy/helm/argentor/
  Chart.yaml
  values.yaml
  templates/
    deployment.yaml    (runAsNonRoot, readOnlyRootFilesystem, seccomp)
    service.yaml
    ingress.yaml
    hpa.yaml           (autoscaling 1→10 replicas)
    pvc.yaml
```

El deployment de Kubernetes incluye:

- **runAsNonRoot:** Los contenedores se ejecutan como usuario no-root
- **readOnlyRootFilesystem:** Sistema de archivos en modo solo lectura
- **Perfil seccomp:** Restricción de llamadas al sistema
- **HPA (Horizontal Pod Autoscaler):** Escalado automático de 1 a 10 réplicas
- **PVC:** Volúmenes persistentes para datos de sesión y caché

16. Métricas del Proyecto

Métrica	Valor
Lenguaje	Rust (core) + WASM (plugins)
Crates	15 (14 workspace + 1 PyO3)
Líneas de código	~130.000
Tests	3.400 exitosos, 0 fallos
Clippy	0 advertencias con `-D warnings`
Proveedores LLM	14
Skills integradas	40+
Endpoints REST	50+
Marcos de cumplimiento	4 (GDPR, ISO 27001, ISO 42001, DPGA)
Patrones de colaboración	6 (Pipeline, MapReduce, Debate, Ensemble, Supervisor, Swarm)
Feature flags	7 (telemetry, sqlite, http-embeddings, registry, browser, docker, client)
Licencia	AGPL-3.0-only
MSRV	1.80
SDKs	Python + TypeScript + PyO3

Canales de Distribución

Canal	Paquete	Comando
crates.io	13 crates	`cargo add argentor-core`
GitHub Releases	Binarios (Linux x86/arm, macOS x86/arm)	Descargar desde releases
PyPI	`argentor-sdk`	`pip install argentor-sdk`
npm	`@argentor/sdk`	`npm install @argentor/sdk`
GHCR	`ghcr.io/fboiero/argentor`	`docker pull ghcr.io/fboiero/argentor`
GitHub Pages	Landing + demos	https://fboiero.github.io/Agentor

---

Generado para Argentor v0.1.0 — Abril 2026

Repositorio: https://github.com/fboiero/Argentor | Licencia: AGPL-3.0-only